

# Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>ChatGPT Coding Architecture — Comprehensive Analysis</b> | <b>2</b> |
| Table of Contents                                           | 2        |
| 1. Overview                                                 | 2        |
| Current State (March 2026)                                  | 3        |
| Key Distinction from CLI Agents                             | 3        |
| 2. Architecture                                             | 3        |
| High-Level System Design                                    | 3        |
| Component Breakdown                                         | 4        |
| 3. Canvas                                                   | 4        |
| What Canvas Is                                              | 4        |
| Core Capabilities                                           | 4        |
| Canvas vs. Claude Artifacts                                 | 5        |
| Limitations of Canvas                                       | 5        |
| 4. Code Interpreter / Advanced Data Analysis                | 5        |
| Sandbox Environment                                         | 5        |
| Pre-installed Libraries (Key Packages)                      | 5        |
| What You Can Do                                             | 5        |
| What You Cannot Do                                          | 6        |
| Code Interpreter vs. Canvas Execution                       | 6        |
| 5. Models for Coding                                        | 6        |
| Currently Available in ChatGPT (March 2026)                 | 6        |
| Recently Retired from ChatGPT                               | 6        |
| API-Only Models Relevant to Coding                          | 6        |
| ChatGPT Context Window Reality                              | 7        |
| Coding Benchmark Comparison (API Models)                    | 7        |
| 6. Custom GPTs for Coding                                   | 7        |
| What Custom GPTs Are                                        | 7        |
| Coding-Focused Community GPTs                               | 7        |
| How They Work for Coding                                    | 7        |
| Limitations vs. Claude Code Skills                          | 7        |
| Key Limitation: Partial Knowledge Retrieval                 | 8        |
| Access Restrictions                                         | 8        |
| 7. Projects Feature                                         | 8        |
| What Projects Are                                           | 8        |
| Core Features                                               | 8        |
| Availability Timeline                                       | 8        |
| Projects vs. Claude Code Project Structure                  | 8        |
| Strengths                                                   | 9        |
| Weaknesses                                                  | 9        |
| 8. Memory System                                            | 9        |
| Architecture (Reverse-Engineered)                           | 9        |
| Key Architectural Decision: No Vector Database              | 9        |
| Coding Preferences Persistence                              | 9        |
| Memory for Coding: Practical Limitations                    | 10       |
| Comparison with Claude Code Memory                          | 10       |
| 9. GitHub Integration                                       | 10       |
| Deep Research GitHub Connector                              | 10       |
| Codex Agent GitHub Integration                              | 10       |
| What ChatGPT Does NOT Have for GitHub                       | 10       |
| 10. Desktop App                                             | 11       |
| macOS App — “Work with Apps”                                | 11       |
| Desktop App vs. Claude Code                                 | 11       |
| Practical Value                                             | 11       |
| 11. API and Function Calling                                | 11       |
| Responses API (Current Standard)                            | 11       |

|                                                           |    |
|-----------------------------------------------------------|----|
| Function Calling for Coding Agents . . . . .              | 12 |
| o3/o4-mini Native Tool Use . . . . .                      | 12 |
| Agents SDK . . . . .                                      | 12 |
| MCP Support . . . . .                                     | 12 |
| Building a Coding Agent with the API . . . . .            | 12 |
| 12. Operator / Computer Use / Codex Agent . . . . .       | 13 |
| Operator (Browser Agent) . . . . .                        | 13 |
| Computer Use (API) . . . . .                              | 13 |
| Codex Agent (In-App) — The Real Coding Autonomy . . . . . | 13 |
| 13. Strengths . . . . .                                   | 14 |
| Where ChatGPT Excels for Coding . . . . .                 | 14 |
| 14. Limitations . . . . .                                 | 14 |
| Fundamental Architectural Limitations . . . . .           | 14 |
| Practical Workflow Limitations . . . . .                  | 14 |
| 15. Pricing . . . . .                                     | 15 |
| Individual Plans (March 2026) . . . . .                   | 15 |
| Team/Business/Enterprise . . . . .                        | 15 |
| API (Pay-as-you-go) . . . . .                             | 15 |
| Coding Value Comparison . . . . .                         | 15 |
| Summary: ChatGPT's Role in the Coding Landscape . . . . . | 16 |

## ChatGPT Coding Architecture — Comprehensive Analysis

**Last updated:** March 2026 **Scope:** ChatGPT web, desktop, and mobile as a coding tool — NOT OpenAI Codex CLI (separate product) **Perspective:** Honest assessment from the standpoint of a Claude Code user examining the competition

---

### Table of Contents

|                                              |
|----------------------------------------------|
| 1. Overview                                  |
| 2. Architecture                              |
| 3. Canvas                                    |
| 4. Code Interpreter / Advanced Data Analysis |
| 5. Models for Coding                         |
| 6. Custom GPTs for Coding                    |
| 7. Projects Feature                          |
| 8. Memory System                             |
| 9. GitHub Integration                        |
| 10. Desktop App                              |
| 11. API and Function Calling                 |
| 12. Operator / Computer Use / Codex Agent    |
| 13. Strengths                                |
| 14. Limitations                              |
| 15. Pricing                                  |

---

### 1. Overview

ChatGPT is a **conversation-first** coding tool. Unlike CLI agents (Claude Code, Codex CLI, Aider) that operate directly in your terminal with file system access, ChatGPT operates through a web/desktop/mobile chat interface. It helps you write, debug, explain, and refactor code — but fundamentally through a dialogue loop where *you* are the executor.

Current State (March 2026)

As of March 2026, ChatGPT’s coding surface area includes:

- **Canvas** — An inline code editor/workspace for iterative editing, review, and execution (Python only)
- **Code Interpreter** — A sandboxed Jupyter/IPython environment for running Python code
- **Projects** — Persistent file storage and custom instructions scoped to a project
- **Memory** — Cross-session recall of coding preferences, languages, and frameworks
- **Work with Apps (macOS)** — Direct reading from and writing to IDEs (VS Code, Xcode, JetBrains)
- **Deep Research GitHub Connector** — Repository analysis and code search via GitHub OAuth
- **Codex (in-app agent)** — Cloud-sandboxed autonomous coding agent that produces pull requests
- **Models** — GPT-5.3 Instant (default), GPT-5.4 Thinking, GPT-5.4 Pro, plus legacy o3/o4-mini in API

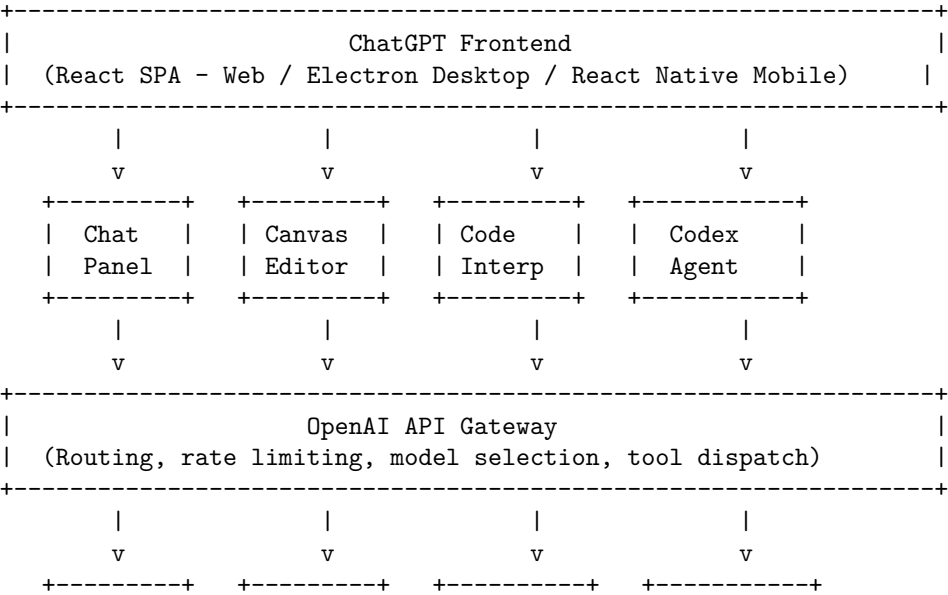
Key Distinction from CLI Agents

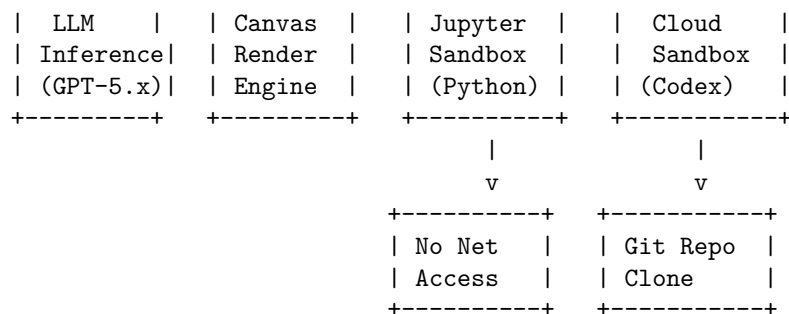
| Aspect             | ChatGPT                               | Claude Code (CLI)                  |
|--------------------|---------------------------------------|------------------------------------|
| Primary interface  | Web/Desktop chat                      | Terminal                           |
| File system access | None (web); limited (macOS app)       | Full local file system             |
| Code execution     | Sandboxed Python only                 | Runs any command on your machine   |
| Git operations     | None directly (Codex agent can PR)    | Full git workflow                  |
| MCP support        | API only (Responses API)              | Native via <code>claude mcp</code> |
| Autonomy           | Conversational; Codex agent for async | Agentic loop with tool use         |
| Context window     | ~32K in chat (model supports more)    | Up to 1M tokens (Opus 4.6)         |

ChatGPT is best understood as a **coding assistant you talk to**, not a coding agent that acts on your behalf. The Codex agent (separate surface within ChatGPT) blurs this line, but the core ChatGPT experience remains conversational.

2. Architecture

High-Level System Design





### Component Breakdown

**Chat Panel:** The standard conversational interface. User sends prompts, model responds with text/code blocks. No code execution. No file access. This is the core ChatGPT experience that 900M+ users interact with.

**Canvas:** A side-panel editor that opens alongside chat. Renders code with syntax highlighting and supports inline editing. The model can make targeted edits to specific sections rather than regenerating entire responses. Backed by a specialized model fine-tuned for edit operations.

**Code Interpreter (Jupyter Sandbox):** A firewalled Linux VM running an IPython kernel. Pre-loaded with 300+ Python packages. No internet access. Files uploaded to the conversation are accessible here. State persists within a session but is destroyed when the session ends.

**Codex Agent:** A separate async agent surface within ChatGPT. Clones your GitHub repo into a cloud sandbox, runs tasks autonomously, and produces PRs. Uses a specialized model (GPT-5.2-Codex or later). Air-gapped — no internet access during execution.

## 3. Canvas

### What Canvas Is

Canvas is ChatGPT’s answer to Claude’s Artifacts — a dedicated, editable workspace that opens as a side panel alongside the chat. It was introduced in October 2024 and exited beta in early 2025, becoming available to all users including the free tier.

### Core Capabilities

**Inline Code Editing** - You can directly edit code in the Canvas panel — it is a real editor, not just a display  
 - Highlight a section and ask ChatGPT to modify just that section - ChatGPT can make surgical edits without regenerating the entire codebase - “Show changes” mode highlights what the model modified

**Coding Shortcuts** (one-click actions in the Canvas toolbar)

| Shortcut         | What It Does                                                                       |
|------------------|------------------------------------------------------------------------------------|
| Review Code      | Provides inline suggestions to optimize and improve code                           |
| Add Logs         | Inserts <code>print()</code> / <code>console.log()</code> statements for debugging |
| Add Comments     | Adds explanatory comments throughout the code                                      |
| Fix Bugs         | Detects and rewrites problematic code sections                                     |
| Port to Language | Translates code to Python, JavaScript, TypeScript, Java, C++, or PHP               |

**Code Execution** - Canvas can execute Python code directly in the browser via the “Execute” button - Output appears in a console panel at the bottom of the Canvas - Currently Python only — other languages cannot be executed  
 - Important limitation: Canvas code execution **cannot read uploaded files** (CSV, etc.) — the traditional Code Interpreter tool can, but Canvas execution cannot

**Version History** - Canvas maintains a version history of edits - Back button restores previous versions - “Show changes” highlights differences between versions

**File Export** - Canvas auto-detects the language and exports with the appropriate file extension (.py, .js, .sql, etc.)  
- Download button for saving code locally

### Canvas vs. Claude Artifacts

| Feature                 | ChatGPT Canvas               | Claude Artifacts              |
|-------------------------|------------------------------|-------------------------------|
| Inline editing          | Yes — direct text editing    | Yes — direct text editing     |
| AI-assisted shortcuts   | Yes (review, fix bugs, port) | Limited (mostly regenerate)   |
| Code execution          | Python only                  | JavaScript (React) in browser |
| Version history         | Yes                          | Yes                           |
| Language porting        | Built-in shortcut            | Manual prompting              |
| HTML/CSS preview        | Yes                          | Yes (full React rendering)    |
| File upload integration | No (cannot read uploads)     | No                            |
| Custom GPT support      | Yes                          | N/A (no equivalent)           |

### Limitations of Canvas

- Code execution is Python-only
- Cannot access files uploaded to the chat
- Cannot install packages during execution
- No terminal/shell access
- Cannot read from or write to local file system
- No integrated testing framework
- Code review suggestions are surface-level — no deep static analysis
- Canvas has a truncation limit on how much code it can display

---

## 4. Code Interpreter / Advanced Data Analysis

### Sandbox Environment

Code Interpreter (originally called “Advanced Data Analysis”) runs a sandboxed IPython kernel inside a Linux container. This is separate from Canvas code execution.

**Environment Specifications:** - **OS:** Ubuntu 20.04.5 LTS - **Architecture:** x86\_64 (not ARM) - **GLIBC:** 2.31  
- **Python:** 3.x with IPython kernel - **Pre-installed packages:** 300+ libraries - **Internet:** None — completely air-gapped - **File upload limit:** 100 MB

### Pre-installed Libraries (Key Packages)

| Category         | Packages                                   |
|------------------|--------------------------------------------|
| Data Analysis    | pandas, numpy, scipy, xarray               |
| Visualization    | matplotlib, seaborn, plotly, bokeh         |
| Machine Learning | scikit-learn, TensorFlow, PyTorch          |
| Image Processing | Pillow, OpenCV                             |
| NLP              | nlTK, spacy (limited)                      |
| General          | requests (but no network), sympy, networkx |

### What You Can Do

- **Data analysis:** Upload CSV/Excel/JSON, analyze with pandas, generate charts
- **File conversion:** Transform between formats (CSV to JSON, image resizing, etc.)
- **Mathematical computation:** Symbolic math, statistics, optimization

- **Visualization:** Create publication-quality charts and graphs
- **Image processing:** Manipulate, annotate, and transform images
- **PDF processing:** Extract text, merge, split PDFs
- **Prototyping:** Write and test Python algorithms iteratively

### What You Cannot Do

- **No internet access:** Cannot fetch URLs, call APIs, or scrape websites
- **No package installation:** Cannot `pip install` (workaround: upload `.whl` files)
- **No persistent state:** Session state is destroyed when the conversation times out
- **No GPU access:** ML model training is CPU-only and very slow
- **No multi-language execution:** Python only (no Node.js, Ruby, Go, Rust, etc.)
- **No shell commands:** Cannot run arbitrary bash commands
- **No file system persistence:** Files created in one session do not carry over
- **Time limits:** Long-running code will be terminated
- **No git operations:** Cannot clone repos or interact with version control

### Code Interpreter vs. Canvas Execution

| Aspect               | Code Interpreter           | Canvas Execution                 |
|----------------------|----------------------------|----------------------------------|
| File access          | Can read uploaded files    | Cannot read uploaded files       |
| Environment          | Full IPython sandbox       | Browser-based lightweight runner |
| Package availability | 300+ pre-installed         | Fewer packages available         |
| Output display       | Inline in chat             | Console at bottom of Canvas      |
| Use case             | Data analysis, computation | Quick script testing             |

## 5. Models for Coding

### Currently Available in ChatGPT (March 2026)

| Model                   | Type                        | Best For                                    | Context Window | Speed                 |
|-------------------------|-----------------------------|---------------------------------------------|----------------|-----------------------|
| <b>GPT-5.3 Instant</b>  | Default for all users       | General coding, fast responses              | ~32K in chat   | Fast                  |
| <b>GPT-5.4 Thinking</b> | Reasoning model             | Complex logic, hard debugging, architecture | ~32K in chat   | Slower (thinks first) |
| <b>GPT-5.4 Pro</b>      | Highest capability          | Hardest problems, long workflows            | ~32K in chat   | Slowest               |
| <b>GPT-5.2 Thinking</b> | Legacy (retiring June 2026) | Backward compatibility                      | ~32K in chat   | Medium                |

### Recently Retired from ChatGPT

As of February 13, 2026, these models are **no longer available in the ChatGPT UI** (still available via API):

- GPT-4o, GPT-4.1, GPT-4.1 mini
- o3, o4-mini
- GPT-5.0, GPT-5.1 (all variants)

### API-Only Models Relevant to Coding

| Model                | Context Window | SWE-bench Verified | Key Strength                        |
|----------------------|----------------|--------------------|-------------------------------------|
| <b>GPT-4.1</b>       | 1M tokens      | 54.6%              | Long-context coding, diff following |
| <b>o3</b>            | 200K tokens    | 69.1%              | Top reasoning for coding            |
| <b>o4-mini</b>       | 200K tokens    | ~66%               | Fast reasoning, cost-efficient      |
| <b>GPT-5.2-Codex</b> | Specialized    | Not published      | Autonomous coding agent model       |
| <b>GPT-5.4</b>       | 128K+          | Not published      | Latest flagship                     |

## ChatGPT Context Window Reality

A critical nuance: while the underlying models (like GPT-4.1) support up to 1M tokens, **ChatGPT's web interface caps effective context at approximately 32K tokens**. The full context windows are only accessible via the API. This means in the ChatGPT UI, you cannot paste an entire large codebase into the conversation — it will be truncated or rejected.

## Coding Benchmark Comparison (API Models)

- **o3**: 69.1% SWE-bench Verified — top OpenAI reasoning model
- **GPT-4.1**: 54.6% SWE-bench Verified — 21.4 percentage points above GPT-4o
- **GPT-4.1**: Doubles GPT-4o's score on Aider's polyglot diff benchmark
- **GPT-4.1**: Beats GPT-4.5 by 8 percentage points on Aider diff
- For comparison: Claude Sonnet 4.5 scores 77.2% on SWE-bench; Claude Opus 4.5 scores 80.9%

## 6. Custom GPTs for Coding

### What Custom GPTs Are

Custom GPTs are user-created specialized ChatGPT configurations with: - Custom system instructions - Uploaded knowledge files (reference documentation, codebases, style guides) - Enabled tools (Code Interpreter, DALL-E, web browsing, Canvas) - Custom Actions (external API calls via OpenAPI spec)

### Coding-Focused Community GPTs

The GPT Store contains thousands of coding-focused GPTs, including: - Language-specific assistants (Python Expert, Rust Helper, etc.) - Framework specialists (React GPT, Django GPT, etc.) - Code review bots - Algorithm explainers - Interview prep assistants

### How They Work for Coding

1. Creator defines system instructions (e.g., "You are a senior Python developer. Always follow PEP 8.")
2. Creator uploads reference files (documentation, coding standards, example code)
3. Creator enables Code Interpreter and/or Canvas
4. Users interact with the GPT, which follows the custom instructions
5. Knowledge files are retrieved via RAG when relevant to the query

### Limitations vs. Claude Code Skills

| Aspect                | Custom GPTs                     | Claude Code Skills                   |
|-----------------------|---------------------------------|--------------------------------------|
| Execution environment | Cloud sandbox (no local access) | Local terminal (full system access)  |
| File system           | Sandboxed uploads only          | Full local file system               |
| Tool access           | Code Interpreter, Web, DALL-E   | bash, read, write, grep, git, MCP    |
| Customization         | Instructions + knowledge files  | Bash scripts with full system access |

| Aspect              | Custom GPTs                             | Claude Code Skills                                     |
|---------------------|-----------------------------------------|--------------------------------------------------------|
| Distribution        | GPT Store (public/private)              | Local <code>.claude/commands/</code> or shared via git |
| Context persistence | Per-conversation (files reset)          | Persistent via local project files                     |
| Knowledge retrieval | Chunked RAG (partial, can miss context) | Direct file reading (complete)                         |

### Key Limitation: Partial Knowledge Retrieval

A significant documented issue: OpenAI's knowledge retrieval system chunks and segments uploaded documents, meaning the GPT only sees a fraction of the knowledge base at any given time. This can lead to incomplete or inaccurate responses when referencing large codebases or documentation sets.

### Access Restrictions

- **Free users:** Can use community GPTs but cannot create custom GPTs
- **Plus/Pro users:** Can create and publish GPTs
- **Team/Enterprise:** Can create private GPTs for the organization

## 7. Projects Feature

### What Projects Are

Projects is a persistent organizational feature in ChatGPT that groups related conversations, files, and custom instructions into a single container. Think of it as a lightweight “workspace” concept.

### Core Features

**Chat Organization** - Group related conversations under a single project - Move existing chats into a project - All chats within a project inherit the project's instructions and file context

**File Management** - Upload files that persist across all conversations in the project - Files stay attached permanently — no re-uploading between sessions

| Tier | Files per Project |
|------|-------------------|
| Free | 5 files           |
| Plus | 25 files          |
| Pro  | 40 files          |

**Custom Instructions (Per-Project)** - Set project-specific system instructions - Example: “This project uses TypeScript, React 19, and Tailwind CSS v4. Follow our ESLint config.” - Instructions apply to all conversations within the project - Stacks with global ChatGPT custom instructions

**Project Sharing** - Available to all tiers (Free, Plus, Pro, Go) - Share projects with team members for collaboration - Available on web, iOS, and Android

### Availability Timeline

- Initially launched for paid users (late 2024)
- Extended to all free users globally on September 3, 2025

### Projects vs. Claude Code Project Structure

| Aspect       | ChatGPT Projects           | Claude Code                   |
|--------------|----------------------------|-------------------------------|
| File storage | Cloud uploads (5-40 files) | Local file system (unlimited) |

| Aspect          | ChatGPT Projects                          | Claude Code                    |
|-----------------|-------------------------------------------|--------------------------------|
| Instructions    | Web UI text field                         | CLAUDE.md files (hierarchical) |
| Context         | Files + instructions injected into prompt | Full repo access + CLAUDE.md   |
| Scope           | Per-project in ChatGPT                    | Per-directory (auto-detected)  |
| Code execution  | Via Code Interpreter only                 | Direct terminal access         |
| Version control | None (file versions not tracked)          | Full git integration           |
| Size limit      | Limited by file count/size                | Limited by disk space          |
| Collaboration   | Project sharing                           | Git-based collaboration        |

## Strengths

- Low barrier to entry — no setup required
- Works across web, desktop, and mobile
- Good for keeping reference docs accessible
- Custom instructions reduce repetitive prompting

## Weaknesses

- Small file limits (especially free tier)
- Cannot reference a local codebase — must upload files
- No directory structure — flat file list
- No automatic sync with local project changes
- No integration with version control

# 8. Memory System

## Architecture (Reverse-Engineered)

ChatGPT’s memory system operates through four distinct layers, as revealed by multiple reverse-engineering analyses in 2025:

**Layer 1: Session Metadata** - Current environment snapshot (device, time, locale) - Injected at session start - Destroyed when session ends - Not saved anywhere

**Layer 2: Saved Memories (Explicit Facts)** - Long-term facts stored as key-value pairs - Accumulated over weeks/months to form a persistent “profile” - Examples: name, job title, preferred language, framework preferences - Created explicitly (“Remember I prefer TypeScript”) or implicitly (model detects a stable preference) - Injected into **every** future prompt as a separate block - User can view, edit, and delete via Settings > Memory

**Layer 3: Chat History Summaries** - Lightweight list of recent conversation summaries - Brief entries — just enough to remind ChatGPT what you discussed recently - Pre-computed summaries injected directly (no vector search, no RAG) - Provides continuity without latency

**Layer 4: Current Conversation Window** - Sliding window of the current conversation’s messages - Subject to the model’s effective context limit (~32K in ChatGPT UI)

## Key Architectural Decision: No Vector Database

Notably, ChatGPT’s memory system does **not** use vector databases or RAG over conversation history. Instead, it pre-computes summaries and injects them directly. This avoids latency but means only summary-level information carries over — fine-grained code details from past conversations are lost.

## Coding Preferences Persistence

In practice, ChatGPT can remember: - Preferred programming languages and frameworks - Coding style preferences (tabs vs. spaces, naming conventions) - Project-specific context (if you tell it) - Technical skill level - Preferred response format (verbose explanations vs. code-only)

## Memory for Coding: Practical Limitations

- Memory entries are **short facts**, not full code snippets or file contents
- Cannot remember an entire codebase structure across sessions
- Cannot remember specific function implementations
- The summary layer captures topics discussed, not code details
- Project files (via Projects feature) are the intended mechanism for persistent code context
- Memory capacity appears to be finite — older memories may be deprioritized

## Comparison with Claude Code Memory

| Aspect         | ChatGPT Memory               | Claude Code Auto-Memory             |
|----------------|------------------------------|-------------------------------------|
| Storage        | Cloud (OpenAI servers)       | Local <b>MEMORY.md</b> files        |
| Format         | Key-value facts + summaries  | Markdown files (human-readable)     |
| User control   | View/delete in Settings      | Direct file editing                 |
| Injection      | All memories in every prompt | Loaded per-project                  |
| Code awareness | Fact-level only              | Can store code patterns, file paths |
| Privacy        | Stored on OpenAI servers     | Local only                          |
| Capacity       | Unclear limits               | Limited by file size                |

---

## 9. GitHub Integration

### Deep Research GitHub Connector

Launched in May 2025, ChatGPT's GitHub connector allows Deep Research to directly analyze repositories.

**How It Works:** 1. User connects their GitHub account via OAuth 2. User enables specific repos for ChatGPT access 3. Deep Research can then search across code, issues, PRs, and docs 4. Produces comprehensive analysis reports

**Capabilities:** - Break down product specs into technical tasks and dependencies - Summarize code structure and patterns - Understand how to implement new APIs using real code examples - Full-text search across authorized repositories - Static analysis and documentation generation

**Security Model:** - Respects GitHub organization permissions - Users only see content they already have access to - Only explicitly shared repos are accessible

**Availability:** - ChatGPT Plus, Pro, and Team - Enterprise and Edu support added later

### Codex Agent GitHub Integration

The Codex agent (separate from Deep Research) has deeper GitHub integration: - Clones authorized repos into cloud sandbox - Creates branches - Makes code changes autonomously - Proposes pull requests - Runs tests within the sandbox

### What ChatGPT Does NOT Have for GitHub

- No real-time sync with local git state
  - No ability to stage/commit/push from the chat interface
  - No branch management from chat
  - No PR review workflow in the main chat UI (Codex handles this separately)
  - No webhook/event-driven responses to repo changes
  - No equivalent to Claude Code's direct **git** command execution
-

# 10. Desktop App

## macOS App — “Work with Apps”

The ChatGPT macOS desktop app (v1.2025.057+) introduced a feature called “Work with Apps” that significantly bridges the gap between a chat tool and a coding agent.

**How It Works:** 1. Press `Option+Space` (or click the menubar icon) to open the Chat Bar 2. ChatGPT reads the content of the **foreground application window** 3. For code editors: includes full content of open editor panes (up to a truncation limit) 4. For terminals: includes the last 200 lines of open terminal panes 5. If you have text selected, ChatGPT focuses on your selection

### Supported Applications:

| Category       | Applications                                                                                             |
|----------------|----------------------------------------------------------------------------------------------------------|
| Code Editors   | Xcode, VS Code, Code Insiders, VSCodium, Cursor, Windsurf                                                |
| JetBrains IDEs | Android Studio, IntelliJ, PyCharm, WebStorm, PHPStorm, CLion, Rider, RubyMine, AppCode, GoLand, DataGrip |
| Text Editors   | TextEdit                                                                                                 |
| Terminals      | Terminal.app, iTerm, Warp, Prompt                                                                        |
| Notes          | Apple Notes                                                                                              |

**Code Editing (Direct Apply):** - When you ask ChatGPT for a code change, it generates a diff - You can review the diff before applying - “Auto-apply” mode applies changes without additional clicks - Writes directly to the open file in the editor

**Availability:** - Plus, Pro, Team: Available since March 2025 - Enterprise, Edu, Free: Rolled out shortly after - Windows: “Coming soon” (as of March 2026)

### Desktop App vs. Claude Code

| Aspect               | ChatGPT Desktop                   | Claude Code                    |
|----------------------|-----------------------------------|--------------------------------|
| IDE integration      | Reads open files, applies diffs   | Reads/writes any file directly |
| Scope                | Foreground window only            | Entire repository              |
| Terminal access      | Reads last 200 lines              | Executes commands directly     |
| Multi-file edits     | One file at a time (visible pane) | Multi-file in single operation |
| Invocation           | <code>Option+Space</code> overlay | Terminal command               |
| Background operation | No                                | Yes (background agents)        |
| Auto-apply           | Optional per-edit                 | Default behavior               |

### Practical Value

The macOS desktop app is genuinely useful for quick coding questions while you are in your editor. The workflow of: select code -> `Option+Space` -> ask question -> apply diff is smooth and faster than switching to a browser. However, it is limited to what is visible in the foreground window and cannot reason about the broader codebase.

# 11. API and Function Calling

## Responses API (Current Standard)

The Responses API (launched March 2025) is OpenAI’s primary API for building coding agents. It replaced the Assistants API (deprecated August 2025, sunseting August 2026).

### Built-in Tools:

| Tool                          | Purpose                                           |
|-------------------------------|---------------------------------------------------|
| <code>web_search</code>       | Real-time web search for up-to-date information   |
| <code>file_search</code>      | Vector store RAG over uploaded documents          |
| <code>code_interpreter</code> | Sandboxed Python execution                        |
| <code>computer_use</code>     | Control a virtual computer (screenshots + clicks) |
| Remote MCP servers            | Connect to any MCP-compatible tool server         |
| Custom functions              | Your own function definitions (JSON schema)       |

**Agentic Loop:** The Responses API supports an agentic loop where the model can call multiple tools within a single API request, chaining tool outputs as inputs to subsequent reasoning steps.

## Function Calling for Coding Agents

Function calling allows developers to build coding agents by defining tools the model can invoke:

```
{
  "name": "read_file",
  "description": "Read contents of a file",
  "parameters": {
    "type": "object",
    "properties": {
      "path": { "type": "string" }
    },
    "required": ["path"]
  }
}
```

The model decides when to call functions, what arguments to pass, and how to interpret results. With `strict: true`, function calls reliably adhere to the schema.

## o3/o4-mini Native Tool Use

The o-series reasoning models (o3, o4-mini) are **trained to use tools natively within their chain of thought**. This means they reason about when and how to use tools as part of their thinking process, rather than tool use being a post-hoc add-on. This is a significant architectural difference from standard function calling.

## Agents SDK

OpenAI's open-source Agents SDK (Python and TypeScript) provides building blocks for: - Tool use orchestration - Agent handoffs (one agent delegates to another) - Guardrails (input/output validation) - Tracing (observability) - Provider-agnostic design (can use non-OpenAI models)

## MCP Support

As of mid-2025, the Responses API supports remote MCP (Model Context Protocol) servers. Developers can connect OpenAI models to external tool servers (Stripe, Shopify, Twilio, etc.) using a few lines of code. However, this is API-only — the ChatGPT web/desktop UI does not expose MCP configuration to end users (unlike Claude Code, where users can add MCP servers via `claude mcp add`).

## Building a Coding Agent with the API

A typical OpenAI coding agent built on the Responses API would:

1. Define tools for file operations (read, write, list, search)
2. Define tools for shell execution (run commands, capture output)
3. Define tools for git operations (status, diff, commit, push)
4. Use the agentic loop to let the model plan and execute
5. Use `code_interpreter` for quick computation/validation

## 6. Use web\_search for documentation lookups

This is essentially what Codex CLI and the Codex app do under the hood. The API gives you full control to build custom variants.

---

## 12. Operator / Computer Use / Codex Agent

### Operator (Browser Agent)

Launched January 2025, Operator is an autonomous browser agent that can: - Navigate websites, fill forms, place orders, schedule appointments - Operates in a virtual machine with limited internet - Controlled by the Computer-Using Agent (CUA) model

**Coding Relevance:** Minimal. Operator is designed for web-based tasks, not coding. It can interact with web-based tools (CI/CD dashboards, cloud consoles) but is not positioned as a coding tool.

### Computer Use (API)

Available via the Responses API, computer use allows a model to: - Take screenshots of a virtual desktop - Click, type, scroll, and navigate - Interact with any GUI application

**Coding Relevance:** Could theoretically interact with IDEs via GUI, but this is slow and fragile compared to direct tool use. Primarily useful for testing web UIs or automating GUI-based workflows.

### Codex Agent (In-App) — The Real Coding Autonomy

The Codex agent, launched in May 2025 and accessible within the ChatGPT interface, is OpenAI's direct answer to agentic coding:

**How It Works:** 1. Connect your GitHub repository 2. Describe a task (feature, bug fix, refactor) 3. Codex clones the repo into a **cloud sandbox** 4. The agent works autonomously: edits files, runs commands, executes tests 5. When done, proposes a **pull request** for your review

**Key Properties:** - **Air-gapped:** No internet access during execution (security) - **Isolated:** Each task runs in its own sandbox - **Asynchronous:** You can leave and come back — it works in the background - **Model:** Uses GPT-5.2-Codex (specialized coding model)

**Availability** (as of March 2026): - Included with ChatGPT Plus, Pro, Business, Enterprise/Edu - Temporarily available to Free and Go users (limited time) - Paid plans get 2x rate limits

### Codex Agent vs. Claude Code:

| Aspect            | Codex Agent                       | Claude Code                        |
|-------------------|-----------------------------------|------------------------------------|
| Environment       | Cloud sandbox (air-gapped)        | Local machine                      |
| Internet access   | None during execution             | Full (via your machine)            |
| Interaction model | Async (submit task, review later) | Interactive (real-time dialogue)   |
| Output            | Pull request                      | Local file changes                 |
| Repo access       | GitHub OAuth                      | Local git repo                     |
| Parallel tasks    | Yes (multiple sandboxes)          | Yes (background agents, worktrees) |
| Customization     | Limited                           | Skills, hooks, MCP, CLAUDE.md      |
| Debugging         | Review PR output                  | Interactive debugging in terminal  |
| Speed             | Minutes to hours                  | Real-time                          |
| Trust model       | Review PR before merge            | Review changes before commit       |

## 13. Strengths

### Where ChatGPT Excels for Coding

- 1. Accessibility and Reach** - 900M+ users — largest user base of any AI coding tool - Works in any browser, no installation required - Mobile app for coding questions on the go - Free tier includes meaningful coding capabilities
  - 2. Rapid Prototyping** - Canvas provides a quick scratchpad for writing and testing Python - Code Interpreter handles data analysis tasks end-to-end - Good for generating boilerplate (REST APIs, React components, database schemas)
  - 3. Explanation and Learning** - Excellent at explaining code concepts, algorithms, and patterns - Conversational format is natural for Q&A about unfamiliar codebases - Custom GPTs provide specialized tutoring experiences
  - 4. macOS Desktop Integration** - Work with Apps reads directly from your IDE - Auto-apply writes changes back to your editor - Smooth workflow for quick, targeted edits
  - 5. Ecosystem Breadth** - Canvas for interactive editing - Code Interpreter for computation - Projects for organization - Memory for preferences - Custom GPTs for specialization - Deep Research for codebase analysis - Codex agent for autonomous tasks - API for building custom tools
  - 6. Data Analysis** - Code Interpreter is excellent for data analysis workflows - Upload CSV -> analyze -> visualize -> download results - Pre-installed libraries cover most data science needs - No setup required — works immediately
  - 7. Conversational Debugging** - Natural back-and-forth for exploring bugs - “Rubber duck” debugging with an AI that can reason about code - Useful for developers who think through problems by talking
- 

## 14. Limitations

### Fundamental Architectural Limitations

- 1. No Local File System Access (Web/Mobile)** - Cannot read your project files directly - Must manually upload or copy-paste code - Cannot traverse directory structures - Cannot read .gitignore, package.json, tsconfig, etc. automatically
- 2. No Terminal/Shell Access** - Cannot run `npm install`, `cargo build`, `pytest`, `make`, etc. - Cannot execute arbitrary commands on your machine - Cannot interact with running processes, servers, or databases - Cannot run your test suite against your actual project
- 3. No Direct Git Integration (in Chat)** - Cannot `git status`, `git diff`, `git commit` - Cannot create branches or manage merges - Cannot interact with CI/CD pipelines - (Codex agent handles some of this, but separately)
- 4. Truncated Context in ChatGPT UI** - Models support up to 1M tokens via API - ChatGPT web UI caps at approximately 32K tokens - Cannot paste or analyze large codebases in conversation - Long conversations lose early context
- 5. No MCP in ChatGPT UI** - MCP is API-only (Responses API) - End users cannot add custom tool servers - No equivalent to Claude Code's `claude mcp add`
- 6. Code Execution is Python-Only** - Code Interpreter: Python only - Canvas execution: Python only - Cannot run TypeScript, Go, Rust, Java, etc. - Cannot test frontend code (no browser rendering in sandbox)
- 7. No Persistent Development Environment** - Code Interpreter state dies with the session - Files created in one conversation don't carry over - Must re-upload context for each new conversation (Projects helps, but limited)
- 8. Sandbox Limitations** - No internet in Code Interpreter or Codex sandbox - Cannot call external APIs during code execution - Cannot install packages (except via .whl upload workaround) - CPU-only (no GPU for ML training)

### Practical Workflow Limitations

- 9. Copy-Paste Workflow Tax** - For web users: code lives in ChatGPT, not in your project - Must manually copy code from Canvas/chat into your editor - Error-prone for multi-file changes - Desktop app reduces this, but limited to foreground window

**10. No Multi-File Reasoning (Web)** - Cannot see your entire project structure - Cannot understand inter-file dependencies - Cannot refactor across multiple files simultaneously - (Desktop app can see open editor panes, but not the whole repo)

**11. No Build/Test Loop** - Cannot run your code, see errors, and fix them iteratively - The “edit -> build -> test -> fix” loop requires manual human steps - Claude Code can do this autonomously in a single agentic loop

**12. Message Rate Limits** - Plus: ~160 GPT-5.2 messages per 3 hours (pre-retirement) - Free: More restrictive limits - Complex coding sessions can exhaust limits quickly

---

## 15. Pricing

### Individual Plans (March 2026)

| Plan    | Price                  | Key Coding Features                                                                                                                                                                     |
|---------|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Free    | \$0                    | GPT-5.3 Instant (limited), Canvas, Code Interpreter (limited), community GPTs, Projects (5 files), Memory                                                                               |
| Go Plus | ~\$35-40/mo<br>\$20/mo | Higher limits than Free, GPT-5-mini, image gen, file analysis<br>GPT-5.3, GPT-5.4 Thinking, Canvas, Code Interpreter, Codex agent, Projects (25 files), Deep Research, GitHub connector |
| Pro     | \$200/mo               | Unlimited GPT-5.4 Pro, highest compute for complex reasoning, 40 files per project, all features                                                                                        |

### Team/Business/Enterprise

| Plan       | Price                         | Key Differentiators                                                                     |
|------------|-------------------------------|-----------------------------------------------------------------------------------------|
| Business   | \$25-30/user/mo               | Admin controls, shared workspace, team GPTs                                             |
| Enterprise | ~\$60+/user/mo (150 seat min) | GPT-5.4, no usage caps, SOC 2, HIPAA BAA, SSO/SCIM, audit logs, org-wide GPT deployment |

### API (Pay-as-you-go)

| Model        | Input (per 1M tokens) | Output (per 1M tokens) |
|--------------|-----------------------|------------------------|
| GPT-4.1      | \$2.00                | \$8.00                 |
| GPT-4.1 mini | \$0.40                | \$1.60                 |
| o3           | \$10.00               | \$40.00                |
| o4-mini      | \$1.10                | \$4.40                 |
| GPT-5.x      | Varies by variant     | Varies by variant      |

### Coding Value Comparison

| Tool                  | Price           | What You Get for Coding                                      |
|-----------------------|-----------------|--------------------------------------------------------------|
| ChatGPT Free          | \$0             | Conversational coding help, Canvas, limited Code Interpreter |
| ChatGPT Plus          | \$20/mo         | Full coding suite including Codex agent                      |
| ChatGPT Pro           | \$200/mo        | Unlimited highest-capability model                           |
| Claude Pro            | \$20/mo         | Claude.ai with Artifacts, Projects                           |
| Claude Code (via Pro) | \$20/mo + usage | Terminal agent with full system access                       |
| Claude Code (via API) | Pay-per-use     | Uncapped terminal agent                                      |
| GitHub Copilot        | \$10-19/mo      | IDE inline completions + chat                                |

## Summary: ChatGPT's Role in the Coding Landscape

ChatGPT is the **most accessible** AI coding tool in the world. With 900M+ users, a free tier, and a conversation-first interface, it is where most developers first experience AI-assisted coding. Its ecosystem (Canvas, Code Interpreter, Projects, Memory, Custom GPTs, Desktop App, Codex Agent, GitHub Connector) is the broadest of any single product.

However, ChatGPT is fundamentally a **conversation tool** for coding, not a **development environment**. The critical gap is the edit-build-test loop: ChatGPT cannot run your code against your real project, cannot execute your test suite, cannot interact with your local tools, and cannot make multi-file changes across a real codebase (web). The desktop app and Codex agent partially address these gaps, but neither provides the seamless, terminal-native experience of a CLI agent.

**Best for:** Quick questions, explanations, prototyping, data analysis, learning, single-file edits via desktop app, autonomous PR generation via Codex agent.

**Not ideal for:** Multi-file refactoring, build/test loops, local tool integration, large codebase navigation, custom workflow automation, MCP-based tool ecosystems.

The market is converging: ChatGPT is adding more agentic capabilities (Codex, desktop app code editing), while CLI agents like Claude Code are adding more conversational polish. By late 2026, the distinction may blur significantly. But as of March 2026, the architectural foundations remain distinct — ChatGPT is a chat-first tool that can sometimes act, while Claude Code is an agent-first tool that can always chat.

---

*Document produced for the Claude Code Features research library. Methodology: Web search across OpenAI official documentation, help center articles, TechCrunch, community forums, reverse-engineering analyses, and benchmark publications. No proprietary or leaked information was used.*